

Step-by-step execution, validation, restoring from backup, and rollback

The concrete order of operations, the checks at each step, how to put a value back from a backed-up JSON file, and what rollback means at each phase.

Assets DC→Cloud migration · attribute remediation series · document 4 of 4

Document set: (1) Data Model & Concepts · (2) Process & Stages · (3) Scripts Reference · (4) Runbook, Recovery & Rollback. Each part is standalone and can be regenerated independently.

1. Prerequisites

- A **sandbox** clone of the DC instance and a Cloud sandbox site. Everything is rehearsed here first.
- A DC **PAT** with Assets read/write; a Confluence DC PAT if pageId→title resolution is needed.
- An Atlassian **API token** + account email for Cloud (Jira and Confluence).
- The five scripts in a working folder; PowerShell 5.1+.

Production discipline. Do not run any `-Execute` against production until the full sequence below has succeeded end-to-end on the sandbox and the before/after backups have been diffed.

2. Step-by-step runbook

1. **Audit (Stage 0).** Run Script 5 for the schema(s). Read the BLOCKERS section. Note every blocker, every object type it sits on (inheritance), any AQL-usage lines, and — with `-CountPopulated` — how many objects are affected.
2. **Build the input list.** From the audit, assemble the `SchemaId, AttributeName, AttributeType` CSV for Script 1 (one row per blocker attribute).
3. **Inventory + backup (Stage 1).** Run Script 1 (v2). Confirm the coverage CSV totals match the value-CSV row count, and that one `_RawValue.txt` exists per object. **This is the point of no data loss** — the backups now exist.
4. **Inspect one value.** Open one `_RawValue.txt` (or re-run Script 1 with `-InspectFirst`). Confirm where the page id lives. If `DC_ConfPageId` is blank for the integration type, adjust before mapping.

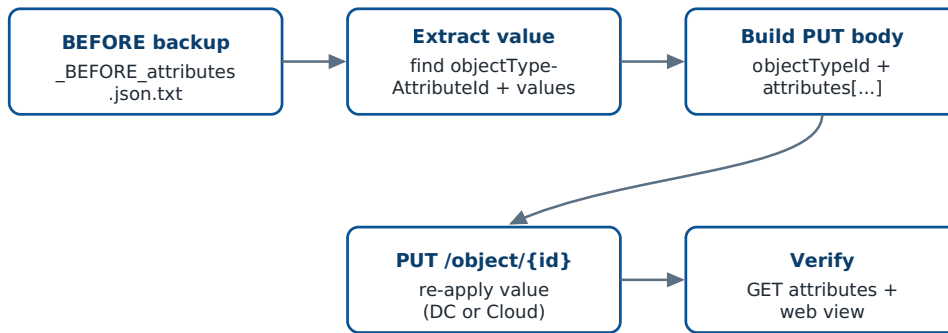
5. **Decide (Stage 2).** For each blocker, choose **clear value** (default) or **delete attribute**. Verify no AQL filter depends on it. Decide the replacement type for Cloud (URL recommended).
6. **Map pages (Stage 3).** Run Script 2. Resolve every non-OK MatchStatus row by hand. The result is your DC→Cloud URL/id bridge.
7. **Pre-migration clear (Stage 4) — dry-run.** Run Script 3 *without* `-Execute` for one object. Read the PLAN and PUT body. Confirm the BEFORE backups were written.
8. **Pre-migration clear — execute.** Re-run with `-Execute`. Diff `BEFORE_attributes` vs `AFTER_attributes`: **only** the target attribute must change. Open the printed web-view link and confirm the value is gone. Repeat for all affected objects (bulk driver once validated).
9. **Confirm JCMA is unblocked.** Re-run the JCMA Assets pre-check. If the attribute type is still listed, delete the attribute in the Assets UI (backups exist) and re-check.
10. **Migrate (Stage 5).** Run the JCMA migration.
11. **Post-migration rebuild (Stage 6) — dry-run then execute.** Run Script 4 for one object without `-Execute`; review the create + set bodies; then `-Execute`. Verify in the Cloud web view that the new attribute holds the Cloud URL.
12. **Validate (Stage 7).** Re-run the inventory against Cloud; reconcile populated counts against the original coverage CSV; spot-check objects. Sign off.

3. Validation checklist

Check	Pass criterion
Inventory completeness	coverage CSV counts sum to value-CSV rows; one backup txt per object
Page map	every input row has a Cloud_ConfPageId, or a documented exception
Clear is surgical	BEFORE vs AFTER differ only in the target attribute
JCMA pre-check	the unsupported attribute type no longer listed
Cloud rebuild	new attribute populated with the correct Cloud URL; resolves in the browser
Reconciliation	Cloud populated count == original DC populated count (minus documented exceptions)

4. Restoring an object from a backed-up JSON

Every write script backs up the full object before changing it. Those files are not just evidence — they are a restore source. The procedure is the same whether you are undoing a wrongly-cleared value in DC or re-applying a value in Cloud: read the value out of the backup, build a minimal update body, PUT it back, verify.



The same flow restores a wrongly-cleared value in DC, or re-applies the intended value in Cloud post-migration.

Figure 4. Restoring / re-applying a value from a backed-up attributes JSON.

1. Locate the value. Open `<Key>_BEFORE_attributes.json.txt`. Find the entry whose `objectTypeAttributeId` is the target attribute; copy its `objectAttributeValues` array.

2. Build the update body. Re-apply just that one attribute:

```

$body = @{
  objectTypeId = <objectTypeId from the BEFORE_object backup>
  attributes = @(
    @{
      objectTypeAttributeId = <AttrId>
      objectAttributeValues = @(
        @{ value = '<value from the backup>' }
      )
    }
  )
}
# DC: PUT {BaseUrl}/rest/assets/1.0/object/{id}
# Cloud: PUT https://api.atlassian.com/jsm/assets/workspace/{workspaceId}/v1/object/{id}
Invoke-RestMethod -Uri $uri -Method Put -Headers $headers `
  -ContentType 'application/json' -Body ($body | ConvertTo-Json -Depth 20)
  
```

3. Verify. GET `/object/{id}/attributes` and confirm the value is back; open the web-view link. Because the update is partial, only the named attribute is touched.

For the Confluence type specifically. A raw DC value re-applied in Cloud will carry a DC page id that no longer resolves. When *re-applying in Cloud*, use the mapped **Cloud** URL (from the page map), not the raw DC value. Use the raw backup verbatim only when restoring back into DC.

5. Rollback by phase

If something goes wrong...	Rollback
during pre-migration clear (DC)	Re-apply the value from <code>BEFORE_attributes</code> (section 4). The DC source is otherwise untouched.
after an attribute was deleted in DC	No undo for the definition; recreate the attribute in the UI, then re-apply values from the backups. This is why deletion is a last resort.

during/after migration	The DC source was kept pristine, so the canonical rollback is to discard the Cloud result and re-migrate. Never treat the migrated Cloud copy as the only source until validation passes.
during Cloud rebuild	Re-running Script 4 is idempotent per object when you pass <code>-ExistingAttrId</code> ; fix the value and set again. Object/type backups capture the prior state.

6. Common edge cases

- **UI key search returns nothing.** `Key = "CMDB-3493"` only matches inside the selected schema and in AQL mode. Use the object id from the inventory CSV, or open the object by id directly.
- **Type-0 "Confluence" attributes.** A link stored in a plain URL/Text field (type 0) is not a JCMA blocker and migrates as-is; it only needs a value rewrite if Confluence itself moved. The inventory keeps these; the audit distinguishes them.
- **Inheritance.** Clearing a parent-declared attribute must cover every child object type that holds a value — the coverage CSV lists them all.
- **Space-key remap.** If CCMA changed space keys, the page map's space clause needs the new key; rows come back NOT_FOUND until corrected.